SCP: Power Evaluation and Study Design for Circadian Biomarker Detection

July 13, 2026

`SCP-package`	*SCP: Power Evaluation and Study Design for Circadian Biomarker Detection*

Description

SCP (Simulation-based Circadian Power) is a framework for sample-size planning in circadian transcriptomic studies. It calibrates gene-level distributions of amplitude, noise, phase, and time-of-day sampling from a user's pilot dataset to capture transcriptome-wide signal heterogeneity, then estimates FDR-controlled power for single-cohort rhythmic-biomarker detection and for two-group differential analyses of rhythmicity (DR), phase (DP), and mesor (DM). A bootstrap layer propagates pilot-estimation uncertainty into confidence intervals on every power estimate. A curated database of public circadian pilots and a Shiny app are bundled for point-and-click planning.

Main functions

Pilot database: `scp_pilots`, `scp_load_pilot`, `scp_pilot_search`.

Calibrate your own pilot: `estCircadianParam`, `estCircadianParam2H`, `estCircadianParamTwoGrou` `prepCircadianData`.

Design and options: `CircadianBioOptions`, `CircadianDesignOptions`, `CircadianAnalysisOption` `CircadianBootstrapOptions`.

Run power analysis: `runSimsSingleCohort`, `runDifferentialPower`, `runBootstrapDesignGrid`, `npower`, `circaPowerApproxN80`, `recommendDesign`.

Plot results: `plotSingleCohortPower`, `plotDiffPower`, `plotBootstrapDesignGrid`.

Detect rhythmicity: `detect_cosinor`.

Simulate and helpers: `simCircadianSingleCohort2H`, `makeAdaptiveRStrata`.

App: `launchShiny`.

Author(s)

Maintainer: Thien Quy Pham <quythien14@gmail.com> (University of Pittsburgh)

See Also

Useful links:

- https://github.com/quythien/SCP
- Report bugs at https://github.com/quythien/SCP/issues

`scp_pilots` *List Bundled SCP Pilot Datasets*

Description

Reads the bundled pilot manifest (`inst/extdata/pilots/manifest.csv`) and returns it as a data.frame. Optionally filters by species and ingestion status so a user can quickly enumerate the pilots they actually have available.

Usage

```
scp_pilots(species = NULL, status = "ingested")
```

Arguments

`species`	Optional character. One or more of `"human"`, `"baboon"`, `"other_primate"`, `"mouse"`, `"rat"`. `NULL` (default) returns all species.
`status`	Character. Filter on the `status` column. Defaults to `"ingested"` (datasets actually present on disk). Use `NULL` or `"all"` to return every row, including `"pending"`, `"verify_failed"`, `"controlled"`, and `"ingest_failed"` entries.

Value

A data.frame with one row per (dataset, tissue, condition) pilot. Columns: `species`, `dataset`, `tissue`, `condition`, `n`, `ngenes`, `design`, `assay`, `sample_type`, `accession`, `citation`, `license`, `file`, `status`.

Examples

```
## Not run:
  scp_pilots()
  scp_pilots(species = "human")
  scp_pilots(status = "all")

## End(Not run)
```

`scp_load_pilot` *Load a Bundled SCP Pilot Dataset*

Description

Loads one standardized pilot object from the bundled pilot database. Each pilot is a pre-fit `CircadianBioOptions` S3 object (fit via the same cosinor pipeline as `estCircadianParam`) and can be passed directly to `runSimsSingleCohort()` or `runDifferentialPower()`.

Usage

```
scp_load_pilot(
  species,
  dataset,
  tissue,
  condition = NULL,
  alpha_pilot = 0.01,
  adjust = c("none", "BH"),
  K = 1,
  paired_sigma = TRUE
)
```

Arguments

`species`	Character, length 1. One of `"human"`, `"baboon"`, `"other_primate"`, `"mouse"`, `"rat"`.
`dataset`	Character, length 1. Manifest `dataset` field (typically GEO accession or short label, e.g. `"GTEx"`, `"GSE160521"`).
`tissue`	Character, length 1. Tissue or region label as recorded in the manifest (e.g. `"Liver"`, `"NAc"`).
`condition`	Character, length 1, optional. Condition arm (e.g. `"Control"`, `"All"`). If `NULL` and the (species, dataset, tissue) triple identifies more than one row, the function errors and prints the available conditions.
`alpha_pilot`	Numeric in `(0, cap]`. Rhythmicity threshold used to re-derive `prop_rhythmic` and the amplitude/phase/sigma distributions from the pilot's stored per-gene rhythm table. Default `0.01` matches the build-time threshold. Only takes effect for pilots rebuilt with a `rhythm_fit` table; older pilots ignore it (with a warning) and return their frozen build-time distributions.
`adjust`	Multiple-testing adjustment applied to the per-gene p-values before thresholding at `alpha_pilot`: `"none"` (raw cosinor p, default) or `"BH"` (Benjamini-Hochberg q-value).
`K`	Harmonic order of the requested pilot: `1` (single-harmonic cosinor, default) or `2` (two-harmonic). For `K = 2` the two-harmonic variant of the pilot (`<file>_2H.rds`) is loaded; an error is raised if no such variant has been built for the pilot.
`paired_sigma`	Logical (default `TRUE`). Keep the per-gene `(A, sigma)` pairing so the simulator draws amplitude and noise from the same pilot gene, preserving the realistic narrow $\tilde{r} = A/\sigma$ distribution (the marginal-power / Fig 1-2 Panel A convention). `FALSE` decouples sigma from amplitude, giving the wide $\tilde{r}$ used by the effect-size-stratified panels (Fig 1-2 Panels B/C).

Value

A `CircadianBioOptions` object with empirical distributions of mesor, log-sigma, amplitude, phase, and proportion rhythmic, ready for downstream simulation.

Examples

```
## Not run:
  bio <- scp_load_pilot("human", "GTEx", "Liver")
  bio <- scp_load_pilot("human", "GTEx", "Liver", alpha_pilot = 0.05)
  bio <- scp_load_pilot("human", "GTEx", "Liver", alpha_pilot = 0.1, adjust = "BH")
  bio <- scp_load_pilot("human", "GSE160521", "NAc", "Control")

## End(Not run)
```

`scp_pilot_search` *Search the SCP Pilot Database*

Description

Convenience filter over the pilot manifest. Combines free-text substring matching against dataset, tissue, citation, and accession fields with optional species and tissue-regex filters and an in-vivo vs cell-line restriction. Useful when a user wants to pick a pilot whose design (tissue, sample size, signal strength) matches their planned study.

Usage

```
scp_pilot_search(
  query = NULL,
  species = NULL,
  tissue_pattern = NULL,
  sample_type = "in_vivo"
)
```

Arguments

`query`	Character, optional. Case-insensitive substring matched against `dataset`, `tissue`, `accession`, `citation`. `NULL` matches everything.
`species`	Character, optional. Restrict to these species.
`tissue_pattern`	Character, optional. Regular expression matched against `tissue` (case-insensitive).
`sample_type`	Character. Default `"in_vivo"`. Use `NULL` to include cell-line pilots as well.

Value

A data.frame: rows of the manifest passing every active filter.

Examples

```
## Not run:
  scp_pilot_search("liver")
  scp_pilot_search(species = "mouse", tissue_pattern = "brain|cortex")
  scp_pilot_search(sample_type = NULL)   # include cell lines

## End(Not run)
```

---

estCircadianParam *Estimate Circadian Parameters from Pilot Data (Single-Cohort)*

Description

Fits a cosinor model per gene, ranks rhythmic genes by p-value, and returns empirical parameter distributions packaged as a `CircadianBioOptions` object suitable for downstream simulation and power analysis.

Usage

```
estCircadianParam(
  data,
  times,
  period = 24,
  min_rhythm_pval = 0.01,
  top_k = 300L,
  prop_DR = 0.15,
  prop_DP = 0.1,
  prop_DM = 0,
  mesor_diff = c(0.5, 2),
  phase_diff = c(-6, 6),
  amp_diff = c(0.5, 2),
  dp_shift_mode = c("fixed", "uniform"),
  dr_amp_scale = 1,
  dr_sigma_scale = 1,
  paired_sigma = FALSE,
  sim.seed = 12345,
  verbose = TRUE
)
```

Arguments

`data`	Numeric matrix of pilot expression (genes x samples). A data.frame is coerced to matrix.
`times`	Numeric vector of sample collection times in hours, length `ncol(data)`. The function does NOT validate units; pass hours (0 to 24 scale).
`period`	Period in the same units as `times` (default 24 hours).
`min_rhythm_pval`	Uncorrected cosinor p-value threshold for the rhythmic-candidate set (default 0.01). Used to compute prop_rhythmic and to select the top-K (K = min(top_k, n_rhythmic)) genes whose amplitudes, phases, and noise drive simulation truth.

`top_k` Number of highest-signal genes (ranked by cosinor p-value) that form the effect-size distribution (amplitude, phase, noise). Default 300. If `top_k` exceeds the number of rhythmic genes, it is capped to that number with a warning. `Inf` or `"all"` uses every rhythmic candidate.

`prop_DR`, `prop_DP`, `prop_DM`
Proportions of differentially-rhythmic, differentially-phased, and differentially-mesor genes (defaults 0.15, 0.10, 0.00). Capped automatically at prop_rhythmic.

`mesor_diff`, `phase_diff`, `amp_diff`
Numeric length-2 ranges for the differential effect-size sampling.

`dp_shift_mode`
One of `"fixed"` or `"uniform"` (default "fixed").

`dr_amp_scale`, `dr_sigma_scale`
Scaling factors for DR-class amplitude and noise distributions (default 1).

`paired_sigma` Logical, default FALSE. If TRUE, joint (A, sigma) sampling preserves pilot correlation in downstream simulations.

`sim.seed` Integer seed for downstream simulation (default 12345).

`verbose` Logical, default TRUE. Print summary at end.

Value

A `CircadianBioOptions` S3 object usable directly with `runSimsSingleCohort()` or `runDifferentialPower()`.

See Also

`estCircadianParamFMM` for FMM-aware pre-screen, `estCircadianParamTwoGroup` for two-group setup, `prepCircadianData` for normalisation upstream.

Examples

```
## Not run:
  set.seed(1)
  n_genes <- 200; n_samples <- 30
  times <- rep(seq(0, 22, by = 2), length.out = n_samples)
  expr  <- matrix(rnorm(n_genes * n_samples), nrow = n_genes)
  for (g in 1:20) expr[g, ] <- expr[g, ] + 2 * cos(2*pi*times/24)
  bio <- estCircadianParam(expr, times, period = 24, verbose = FALSE)

## End(Not run)
```

`estCircadianParam2H` *Estimate CircadianBioOptions Under the Two-Harmonic Cosinor Model*

Description

Two-harmonic analog of `estCircadianParam`. Fits the regression

$$y = \mu + a_1 \cos(\omega_0 t) + b_1 \sin(\omega_0 t) + a_2 \cos(2\omega_0 t) + b_2 \sin(2\omega_0 t) + \epsilon$$

to each gene by OLS, tests the joint rhythmicity null $H_0 : a_1 = b_1 = a_2 = b_2 = 0$ on 4 d.f., and returns a `CircadianBioOptions` object whose 5-way joint empirical distribution $F_{(\mu, A_1, \phi_1, A_2, \phi_2, \sigma)}$ is preserved by gene-paired index draws downstream.

The returned object carries the standard 1-harmonic fields (`amplitude`, `phase`, `sigma_rhythmic`) holding the 1st-harmonic $A_{g,1}$, $\phi_{g,1}$, and σ_g respectively, and three extra fields attached to the list after construction:

- `amplitude2`: 2nd-harmonic amplitudes $A_{g,2}$
- `phase2`: 2nd-harmonic acrophases $\phi_{g,2}$ in hours $[0, period/2)$
- `paired_2h`: `TRUE` flag (downstream simulators read this to switch on the 2-harmonic generative model).

These extras live alongside `$diagnostics`; the constructor is not modified.

Usage

```
estCircadianParam2H(
  data,
  times,
  period = 24,
  min_rhythm_pval = 0.01,
  top_k = 500,
  prop_DR = 0.15,
  prop_DP = 0.1,
  prop_DM = 0,
  mesor_diff = c(0.5, 2),
  phase_diff = c(-6, 6),
  amp_diff = c(0.5, 2),
  dp_shift_mode = c("fixed", "uniform"),
  dr_amp_scale = 1,
  dr_sigma_scale = 1,
  paired_sigma = FALSE,
  sim.seed = 12345,
  verbose = TRUE
)
```

Arguments

`data`	Numeric matrix of pilot expression (genes x samples).
`times`	Numeric vector of sample collection times in hours, length `ncol(data)`.
`period`	Period in the same units as `times` (default 24 hours).
`min_rhythm_pval`	P-value threshold for the joint 4-d.f. F-test (default 0.01). Genes with p < this define the candidate set.
`top_k`	Cap on number of genes contributing to the joint $(A_1, \phi_1, A_2, \phi_2, \sigma)$ distribution (default 500), ranked by F-test p-value.

`prop_DR`, `prop_DP`, `prop_DM`
: Differential proportions (defaults 0.15, 0.10, 0.00). Forwarded unchanged to `CircadianBioOptions`; note the K=2 extension itself is single-cohort only.

`mesor_diff`, `phase_diff`, `amp_diff`
: Differential effect-size ranges.

`dp_shift_mode`
: "fixed" or "uniform" (default "fixed").

`dr_amp_scale`, `dr_sigma_scale`
: DR scale factors (default 1).

`paired_sigma` Logical (default FALSE). Forwarded to `CircadianBioOptions`.

`sim.seed` Integer seed (default 12345).

`verbose` Logical (default TRUE). Print diagnostic summary.

Value

A `CircadianBioOptions` object with extra fields `$amplitude2`, `$phase2`, `$paired_2h`, and a `$diagnostics` list summarising the 2H fit (median A2/A1 ratio, fraction of genes with significant 2nd-harmonic block, etc.).

Methodological notes

1. Pilot summary is a 5-way joint empirical distribution. When the simulator draws a rhythmic gene, the same index j is used across $(A_1, \phi_1, A_2, \phi_2, \sigma)$, preserving the cross-parameter dependence observed in the pilot.
2. No 2nd-harmonic significance threshold is applied: ALL top-K genes passing the joint 4-d.f. F-test contribute, even those whose A_2 is essentially noise. The empirical F_{A_2} will have a noise spike near zero, which is fine.
3. K=2 extension is single-cohort only – the differential simulator (`simCircadianDiff`) is not extended.

See Also

`estCircadianParam` for the 1-harmonic case, `simCircadianSingleCohort2H` for the matching simulator.

---

estCircadianParamTwoGroup

Estimate a two-group circadian pilot for differential power

Description

Fits cosinor models to two groups of pilot data and assembles the two-group pilot summary used by the differential power simulator, encoding differential rhythmicity (DR), differential phase (DP), and differential MESOR (DM) structure.

Usage

```
estCircadianParamTwoGroup(
  data_1,
  data_2,
  times_1,
  times_2,
  period = 24,
  min_rhythm_pval = 0.01,
  top_k = 300L,
  phase_shift_threshold = 2,
  prop_DM = NULL,
  mesor_diff = NULL,
  dp_shift_mode = c("uniform", "fixed"),
  paired_sigma = FALSE,
  sim.seed = 12345,
  verbose = TRUE
)
```

Arguments

`data_1, data_2`	Gene-by-sample expression matrices for the two groups.
`times_1, times_2`	Numeric collection times for each group.
`period`	Period in hours (default 24).
`min_rhythm_pval`	Marginal p-value screen defining the rhythmic pilot set.
`top_k`	Number of highest-signal genes (per group, ranked by cosinor p-value) forming each group's effect-size distribution (default 300). Capped (with a warning) to a group's rhythmic-gene count if larger; `Inf` or `"all"` uses every rhythmic candidate.
`phase_shift_threshold`	Minimum phase difference (hours) for the DP class.
`prop_DM`	Optional target proportion of differential-MESOR genes.
`mesor_diff`	Optional MESOR difference for the DM component.
`dp_shift_mode`	Differential-phase shift model, "uniform" or "fixed".
`paired_sigma`	Preserve the paired (amplitude, noise) relationship across groups.
`sim.seed`	Random seed.
`verbose`	Print progress.

Value

A two-group `CircadianBioOptions` pilot summary.

`prepCircadianData` *Prepare expression data for circadian power analysis*

Description

Converts raw expression data into the standard (matrix, times) format expected by `estCircadianParam()`, `estCircadianParamTwoGroup()`, and `runBootstrapDesignGrid()`.

The pipeline always expects:

- `data`, numeric matrix, **genes x samples**, log2-scale
- `times`, numeric vector, hours in [0, 24), same length as `ncol(data)`

This function handles the three most common raw formats:

"counts" Raw integer counts (e.g. featureCounts, STAR). Converts to log2(CPM + 1): `log2(sweep(x, 2, colSums(x), "/") * 1e6 + 1)`.

"cpm" Raw CPM values (e.g. edgeR `cpm()`, baboon data). Converts to log2(CPM + 1).

"log2" Already log2-scale (e.g. GSE54651, Seney ACC). Wraps with `data.matrix()` only; no numeric transformation.

Usage

```
prepCircadianData(
  expr,
  times,
  input_type = c("counts", "cpm", "log2"),
  pheno = NULL,
  sample_col = NULL,
  time_col = NULL,
  period = 24,
  verbose = TRUE
)
```

Arguments

`expr`	A matrix, data.frame, or file path (CSV/TSV) of expression values. Rows = genes, columns = samples. Row names used as gene IDs.
`times`	Numeric vector of time-of-day values (hours, 0-24) for each sample, OR a character column name in `pheno` to extract times from.
`input_type`	One of `"counts"`, `"cpm"`, or `"log2"`. Default `"counts"`.
`pheno`	Optional data.frame of sample metadata. Must have the same column order as `expr` if `times` is a column name string.
`sample_col`	Optional: name of the column in `pheno` whose values match `colnames(expr)` for alignment. If NULL, assumes columns are already in the same order as rows of `pheno`.
`time_col`	Name of the TOD column in `pheno` (used when `times` is a column name string).
`period`	Circadian period in hours. Default 24. Used only for validation.
`verbose`	Print a short summary of what was done. Default TRUE.

Value

A list with:

`data`	Numeric matrix (genes x samples), log2-scale
`times`	Numeric vector of hours (0-24), length == ncol(data)
`n_genes`	Number of genes
`n_samples`	Number of samples
`input_type`	The `input_type` used

Examples

```
## Not run:
# From raw counts matrix
prep <- prepCircadianData(counts_matrix, times = tod_vector, input_type = "counts")
bio  <- estCircadianParam(prep$data, prep$times)

# From data.frame with metadata
prep <- prepCircadianData(
  expr       = expr_df,
  times      = "TOD",
  input_type = "log2",
  pheno      = pheno_df,
  sample_col = "SampleID"
)

# Baboon raw CPM (load gives a data.frame)
prep <- prepCircadianData(baboon_df, times = tod_vector, input_type = "cpm")

## End(Not run)
```

`CircadianBioOptions` *Create Biology + Differential Options*

Description

Create Biology + Differential Options

Usage

```
CircadianBioOptions(
  ngenes = 5000,
  prop_rhythmic = NULL,
  period = 24,
  lBaselineExpr = NULL,
  lBaselineExpr2 = NULL,
  lOD = NULL,
  lOD2 = NULL,
  amplitude = NULL,
  amplitude2 = NULL,
  sigma_rhythmic = NULL,
```

```
  paired_sigma = FALSE,
  paired_omega = FALSE,
  paired_alpha = FALSE,
  omega_rhythmic = NULL,
  alpha_rhythmic = NULL,
  omega_dist = NULL,
  alpha_dist = NULL,
  cts = NULL,
  cts2 = NULL,
  phase = "uniform",
  prop_DR = 0.15,
  prop_DP = 0.1,
  prop_DM = 0,
  phase_diff = c(-6, 6),
  amp_diff = c(0.5, 2),
  mesor_diff = c(0.5, 2),
  dp_shift_mode = c("fixed", "uniform"),
  dr_amp_scale = 1,
  dr_sigma_scale = 1,
  alpha_pilot = NULL,
  sim.seed = 12345
)
```

Arguments

`ngenes`	Number of genes.
`prop_rhythmic`	Proportion of rhythmic genes. If `NULL` and a pilot dataset name is supplied to `lBaselineExpr`, the pilot's estimated proportion is used.
`period`	Circadian period in hours.
`lBaselineExpr`	Log baseline expression. Can be a numeric scalar (constant for all genes), a numeric vector (resampled to `ngenes`), a function `f(n)` returning `n` values, or a character string naming a built-in pilot dataset stored in `data/`. No default, must be supplied.
`lBaselineExpr2`	As `lBaselineExpr` but for group 2 (differential analyses only). `NULL` uses the same distribution as group 1.
`lOD`	Log over-dispersion (noise). Same forms accepted as `lBaselineExpr`. No default, must be supplied.
`lOD2`	As `lOD` but for group 2. `NULL` shares group 1 values.
`amplitude`	Amplitude distribution for rhythmic genes. Same forms accepted as `lBaselineExpr`. No default, must be supplied.
`amplitude2`	As `amplitude` but for group 2.
`sigma_rhythmic`	Optional numeric vector of per-gene noise values for rhythmic genes (same length as `amplitude`). When provided, amplitude and sigma are drawn jointly to preserve pilot A-sigma correlation.
`paired_sigma`	Logical. If `TRUE` and `sigma_rhythmic` is supplied, expand `amplitude` and `sigma_rhythmic` jointly by the same resampled gene index so their

pilot-estimated pairwise correlation is preserved; if FALSE (default), amplitude and sigma are expanded independently.

paired_omega Logical. If TRUE and omega_rhythmic is supplied (same length as amplitude), expand omega using the same shared gene-index used for amplitude/sigma so all three are paired per gene.

paired_alpha Logical. Same as paired_omega for alpha_rhythmic.

omega_rhythmic
Optional numeric vector of per-gene FMM omega values from a pilot FMM fit (same length as amplitude). Used when paired_omega = TRUE; ignored when omega_dist is also set.

alpha_rhythmic
Optional numeric vector of per-gene FMM alpha values in radians from a pilot FMM fit (same length as amplitude).

omega_dist Optional list specifying a distribution from which omega is drawn at simulation time, e.g. list(family="beta", a=1, b=5) or list(family="fixed", value=0.7). Overrides omega_rhythmic when both are provided.

alpha_dist Optional list specifying a distribution for alpha, list(family="normal", mean=0, sd_hours=2). Internally sd_hours is converted to radians via sd_rad = sd_hours * 2*pi/24. When alpha_rhythmic is also provided and lengths match, the perturbation is added to each gene's empirical alpha (alpha_g = alpha_rhythmic[g] + N(0, sd_rad^2)). When alpha_rhythmic is missing, alpha_g = N(0, sd_rad^2).

cts Numeric vector of sample collection times for passive design.

cts2 As cts for group 2.

phase Phase distribution for rhythmic genes. "uniform" (default) or the same forms as lBaselineExpr.

prop_DR Proportion with differential rhythmicity.

prop_DP Proportion with differential phase.

prop_DM Proportion with differential mesor (mean shift, both groups rhythmic).

phase_diff Range of phase shift for DP genes c(min, max).

amp_diff Range of amplitude ratio (unused; retained for interface compatibility).

mesor_diff Range of mesor shift for DM genes c(min, max) (additive, log-scale units).

dp_shift_mode
"fixed" (use phase_diff[2]) or "uniform" (sample uniformly within phase_diff range).

dr_amp_scale Scale factor for amplitude (A) to adjust DR strength.

dr_sigma_scale
Scale factor for sigma to adjust DR strength.

alpha_pilot Rhythmicity pre-screen threshold used to define the pilot's rhythmic gene set (the value at which prop_rhythmic and the effect-size distribution were calibrated). Stored on the object for provenance and reporting; default NULL.

sim.seed Random seed.

Value

Object of class "CircadianBioOptions".

`CircadianDesignOptions` *Create Study Design Options*

Description

Create Study Design Options

Usage

```
CircadianDesignOptions(
  sample_sizes = c(10, 20, 40, 60, 80, 100),
  nsims = 100,
  design = c("active", "passive"),
  cts = NULL,
  B_values = NULL,
  test_types = c("DR", "DP", "DM"),
  omega = 1,
  beta = pi
)
```

Arguments

`sample_sizes`	Vector of sample sizes per group
`nsims`	Number of simulation replicates
`design`	"active" or "passive"
`cts`	Time-of-day distribution for passive design (numeric vector)
`B_values`	Optional vector of time-point counts (B) to sweep, in addition to `sample_sizes`; NULL runs a single B implied by `cts`.
`test_types`	Character vector of tests to run ("DR", "DP", "DM")
`omega`	FMM waveform shape parameter in (0, 1]. `omega = 1` (default) is the pure cosinor path; `omega < 1` simulates a non-sinusoidal waveform via `simCircadianFMM()`. `omega = 0` is not allowed.
`beta`	FMM orientation parameter (peak-location offset, default `pi`); only used when `omega < 1`, ignored for the cosinor path.

Value

Object of class "CircadianDesignOptions"

`CircadianAnalysisOptions` *Create Analysis & Reporting Options*

Description

Create Analysis & Reporting Options

Usage

```
CircadianAnalysisOptions(
  alpha = 0.05,
  p.adjust.method = "BH",
  parallel.ncores = 1,
  amp.cutoff = 0,
  target_effect = 0.1,
  fdr_thresholds = c(0.01, 0.05, 0.1, 0.2),
  reference_n = 60,
  r_strata = c(0, 0.25, 0.5, 0.75, 1, 1.25, 1.5, 1.75, 2, 2.5, 3, 3.5, 4, 4.5, 5
  strata_labels = NULL,
  phase_shifts = c(0, 0.5, 1, 2, 4, 6, 8, 10, 12),
  DCmethod = "DCP"
)
```

Arguments

`alpha`	Significance level for DCP pipeline
`p.adjust.method`	Multiple testing correction method
`parallel.ncores`	Number of cores for parallel DCP
`amp.cutoff`	Amplitude cutoff for DCP_Rhythmicity
`target_effect`	Minimum effect size to count as "interesting"
`fdr_thresholds`	FDR thresholds for power curves
`reference_n`	Reference sample size for Panel B / phase shift plots
`r_strata`	Breakpoints for A/sigma stratification
`strata_labels`	Labels for strata (auto-generated if NULL)
`phase_shifts`	Phase shift magnitudes to sweep (for sensitivity analysis)
`DCmethod`	Differential-rhythmicity detector to report against, one of "DCP" (default) or "CircaCompare".

Value

Object of class "CircadianAnalysisOptions"

`CircadianBootstrapOptions` *Create Bootstrap Design Grid Options*

Description

Create Bootstrap Design Grid Options

Usage

```
CircadianBootstrapOptions(
  design_vector,
  B_values = c(4, 6, 8, 12, 24),
  m_values = NULL,
  N_values = NULL,
  nboot = 200,
  nsims_inner = 20,
  design = c("active", "passive"),
  seed = 42
)
```

Arguments

`design_vector`	Ordered time points to sweep over (length >= max(B_values))
`B_values`	Number of distinct time points to test
`m_values`	Replicates per time point (if NULL, derived as round(N/B))
`N_values`	Total per-group sample sizes to test (if NULL, derived as B*m)
`nboot`	Number of bootstrap parameter draws (outer uncertainty loop)
`nsims_inner`	Simulations per bootstrap draw
`design`	"active" or "passive"
`seed`	Random seed

Value

Object of class "CircadianBootstrapOptions"

`runSimsSingleCohort` *Simulation-Based Power for Single-Cohort Rhythmicity Detection*

Description

Extends the closed-form CircaPower approach by running full simulations: for each sample size and simulation replicate, generates data from the empirical pilot parameter distributions, applies the cosinor F-test per gene, applies BH correction, and aggregates empirical power and FDR.

Usage

```
runSimsSingleCohort(
  bio.opts,
  design.opts,
  analysis.opts,
  method = "cosinor",
  K = 1L,
  harmonics = c(0, 0),
  verbose = TRUE,
  mc.cores = 1L,
  progress = NULL
)
```

Arguments

bio.opts	CircadianBioOptions – pilot parameter distributions. Build with estCircadianParam().
design.opts	CircadianDesignOptions – sample sizes, nsims, design.
analysis.opts	CircadianAnalysisOptions – alpha, p.adjust.method.
method	Detection method: "DCP" (1-harmonic cosinor F-test), "JTK", "RAIN", "MH" (multi-harmonic), or "FMM" (K-harmonic F-test motivated by the FMM Fourier expansion; default detector for non-cosinor signals).
K	Integer. Number of harmonics for method = "FMM" (default 2). K=1 is equivalent to DCP; K=2 captures the 1st and 2nd Fourier harmonics; K=3+ for very sharp peaks.
harmonics	Numeric length-2: c(alpha2, alpha3) harmonic coefficients (default c(0,0)).
verbose	Print progress (default TRUE).
mc.cores	Parallel cores (default 1).
progress	Optional callback function(sample_size_index, n_sample_sizes) invoked after each sample size completes (used by the Shiny app to advance its progress bar). NULL (default) disables the callback.

Details

Simulation-based alternative to the closed-form CircaPower formula: works for any design (active or passive) and any pilot parameter distribution.

Value

List with:

marginal_power	Matrix [sample_sizes x nsims]
marginal_FDR	Matrix [sample_sizes x nsims]
marginal_TD	Matrix [sample_sizes x nsims]
marginal_FD	Matrix [sample_sizes x nsims]
marginal_alpha	Matrix [sample_sizes x nsims] – empirical type-I error
pvalues	Array [sample_sizes x ngenes x nsims] – raw p-values

`r_values_list`
: List[[sample_size]][[sim]] – per-gene A/sigma

`strat_power`
: Array [sample_sizes x n_strata x nsims]

`strat_TD`
: Array [sample_sizes x n_strata x nsims]

`strat_FD`
: Array [sample_sizes x n_strata x nsims]

`strat_n_targets`
: Array [sample_sizes x n_strata x nsims]

`strat_n_nulls`
: Array [sample_sizes x n_strata x nsims]

`strat_FDR`
: Array [sample_sizes x n_strata x nsims]

`strat_alpha`
: Array [sample_sizes x n_strata x nsims]

`r_strata`
: Stratum breakpoints (from analysis.opts)

`strata_labels`
: Stratum labels

`n0_circapower`
: CircaPower n80 estimate from pilot median r

`sample_sizes`
: Vector of sample sizes

`nsims`
: Number of simulation replicates

Examples

```
## Not run:
bio  <- estCircadianParam(expr, times)
dopt <- CircadianDesignOptions(sample_sizes = c(20, 40, 80), nsims = 50)
aopt <- CircadianAnalysisOptions(alpha = 0.05)
res  <- runSimsSingleCohort(bio, dopt, aopt)
rowMeans(res$marginal_power)  # mean power at each N

## End(Not run)
```

`runDifferentialPower` *Run two-group differential circadian power analysis*

Description

Sweeps N x alpha2 x alpha3 x method x test_type and returns a tidy power table. Automatically calls plot() and npower() unless plot=FALSE.

Usage

```
runDifferentialPower(
  bio.opts,
  design.opts,
  analysis.opts,
  methods = "DCP",
  test_types = c("DR", "DP", "DM"),
  alpha2 = 0,
```

```
  alpha3 = 0,
  mc.cores = 1L,
  plot = TRUE,
  output_file = NULL,
  verbose = TRUE
)
```

Arguments

bio.opts	CircadianBioOptions from estCircadianParamTwoGroup()
design.opts	CircadianDesignOptions (sample_sizes, design, cts)
analysis.opts	CircadianAnalysisOptions
methods	Any of "DCP","CircaCompare"
test_types	Any of "DR","DP","DM" (silently NA if method lacks support)
alpha2	Scalar or vector swept for both groups
alpha3	Scalar or vector swept for both groups
mc.cores	Parallel cores
plot	Auto-plot on completion
output_file	PDF path (NULL = screen)
verbose	Print progress

Value

List (class `SCPDiffResult`) from `runSimsDiff()`: `pval_DR`, `fdr_DR`, `pval_DP`, `fdr_DP`, `pval_DM`, `fdr_DM` – 3-D arrays `[ngenes x n_sizes x nsims]`; plus `diff_type`, `effectsize`, `sample_sizes`, `nsims`. Pass to `plotDiffPower()` or `npower(..., endpoint="DR")`.

---

runBootstrapDesignGrid

Run bootstrap design grid simulation

Description

Estimates uncertainty in power across (N, B) design configurations by bootstrapping parameter sets from pilot data.

Usage

```
runBootstrapDesignGrid(
  pilot_data,
  pilot_times,
  boot.opts,
  analysis.opts,
  bio_diff.opts,
  pilot_data_2 = NULL,
  pilot_times_2 = NULL,
  mode = NULL,
```

```
  methods = "DCP",
  test_types = c("DR", "DP", "DM"),
  alpha2 = 0,
  alpha3 = 0,
  min_rhythm_pval = 0.01,
  resample = c("subject", "gene"),
  verbose = TRUE,
  mc.cores = 1L
)
```

Arguments

pilot_data	Genes x samples matrix (pilot experiment).
pilot_times	Numeric vector of sample time points, length = ncol(pilot_data).
boot.opts	CircadianBootstrapOptions from CircadianBootstrapOptions(). Must set B_values to a single value for passive designs.
analysis.opts	CircadianAnalysisOptions.
bio_diff.opts	CircadianBioOptions from estCircadianParam(). Required even in mode = "single", differential proportion fields are read from here.
pilot_data_2	Optional group-2 pilot matrix for mode = "differential".
pilot_times_2	Optional group-2 pilot times.
mode	"single" or "differential".
methods	Detection method(s): "DCP" (default), "JTK", "RAIN".
test_types	Endpoints for differential mode: any of "DR", "DP", "DM".
alpha2	Second-harmonic coefficient (default 0).
alpha3	Third-harmonic coefficient (default 0).
min_rhythm_pval	P-value threshold for pilot rhythmicity (default 0.01).
resample	Resampling scheme for the outer bootstrap. "subject" (default) is the Efron nonparametric bootstrap over pilot subjects: each draw resamples subject columns with replacement (carrying their collection times) and refits the pilot summary, matching the Section 2.3 description. "gene" is the legacy scheme that fits once and resamples gene rows of the fit; retained for backward compatibility and diagnostics only.
verbose	Print progress (default TRUE).
mc.cores	Parallel cores for inner simulation loop (default 1).

Value

Named list with 15 elements:

N_values	Candidate sample sizes tested.
B_values	Candidate time-point counts tested.
m_matrix	Matrix of total samples (N x B).
nboot	Number of bootstrap replicates.

`design` Design type (`"active"` or `"passive"`).

`test_types` Endpoints evaluated.

`fdr_threshold` FDR threshold used.

`boot_power` 4-D array `[nboot x n_N x n_B x n_tests]`.

`power_mean` Mean power array `[n_N x n_B x n_tests]`.

`power_se` Bootstrap SE array, same shape as `power_mean`.

`power_ci_lo` 2.5th percentile CI array.

`power_ci_hi` 97.5th percentile CI array.

`optimal_B` Integer vector (length n_N): B with highest mean power at each N.

`optimal_B_ci_lo` Bootstrap 2.5th percentile of optimal B.

`optimal_B_ci_hi` Bootstrap 97.5th percentile of optimal B.

---

npower *Find the sample size needed to achieve a target genome-wide power*

Description

Works on output from either `runSimsSingleCohort()` or `runSimsDiff()`. Recomputes marginal power at the requested FDR from raw simulation output, then returns the smallest grid N that meets the target (`n_grid`) and, when `interpolate = TRUE` (the default), a linearly interpolated estimate between the two bracketing grid points (`n`).

Usage

```
npower(
  res,
  target_power = 0.8,
  fdr = 0.05,
  endpoint = NULL,
  p.adjust.method = "BH",
  interpolate = TRUE
)
```

Arguments

`res`	Output from `runSimsSingleCohort()` or `runSimsDiff()`.
`target_power`	Numeric in (0, 1). Target marginal power (default 0.80).
`fdr`	FDR threshold (default 0.05).
`endpoint`	For `runSimsDiff()` output: one of `"DR"`, `"DP"`, `"DM"`. Ignored for single-cohort output.
`p.adjust.method`	Multiple-testing correction (default `"BH"`). Only applied for single-cohort output (differential uses pre-computed FDR arrays).
`interpolate`	Logical. If `TRUE` (default), linearly interpolates between the two adjacent grid points to return a more precise `n` estimate. Set to `FALSE` to revert to the grid-snapped `n_grid` value (backward-compatible).

Value

An object of class "npower" (a list) with:

n	Recommended sample size: n_interp when interpolate = TRUE, otherwise n_grid. NA if target power is never reached.
n_grid	Smallest sample size in the simulation grid achieving target power, or NA.
n_interp	Linearly interpolated sample size between the two bracketing grid points (a fractional N rounded up). NA when interpolate = FALSE or when target power is never reached.
power	Named numeric vector: mean power at each grid sample size.
target_power, fdr, endpoint	Echo of inputs.

---

circaPowerApproxN80

Estimate n for 80 percent power using CircaPower at median pilot r

---

Description

Estimate n for 80 percent power using CircaPower at median pilot r

Usage

```
circaPowerApproxN80(
  bio.opts,
  alpha = 0.05,
  target_power = 0.8,
  n_search = seq(5, 500, by = 5)
)
```

Arguments

bio.opts	CircadianBioOptions with amplitude and sigma_rhythmic.
alpha	Significance level (default 0.05).
target_power	Target power (default 0.80).
n_search	Integer vector of candidate sample sizes.

Value

Single integer – smallest n achieving target_power, or NA if not found.

`recommendDesign` *B vs m design study – analytical + simulation*

Description

Full B vs m study orchestrator. Three sequential steps:

Usage

```
recommendDesign(
  bio.opts,
  design.opts,
  analysis.opts,
  methods = c("DCP", "JTK", "RAIN", "MH"),
  target_power = 0.8,
  mode = c("single", "differential"),
  run_simulation = TRUE,
  prior_result = NULL,
  alpha2 = 0,
  alpha3 = 0,
  mc.cores = 1L,
  plot = TRUE,
  output_file = NULL,
  verbose = TRUE
)
```

Arguments

`bio.opts`	CircadianBioOptions from estCircadianParam() or estCircadianParamTwoGroup()
`design.opts`	CircadianDesignOptions with sample_sizes and B_values
`analysis.opts`	CircadianAnalysisOptions
`methods`	Detection methods to compare. Single-cohort: c("DCP","JTK","RAIN","MH"). Differential: c("DCP","CircaCompare","LimoRhyde","DODR").
`target_power`	Target power for recommendation (default 0.80)
`mode`	"single" or "differential" (default "single")
`run_simulation`	TRUE = run full simulation after analytical step. FALSE = analytical (CircaPower) only.
`prior_result`	A previous runSingleCohortGrid()/runDifferentialPower() result to reuse instead of re-running simulation.
`alpha2`	2nd-harmonic deviation (scalar or vector)
`alpha3`	3rd-harmonic deviation (scalar or vector)
`mc.cores`	Parallel cores for simulation step
`plot`	Auto-plot final recommendation
`output_file`	PDF path (NULL = screen)
`verbose`	Print progress

Details

1. Print method guidance table (printMethodGuidance) 2. Analytical: CircaPower closed-form power at each N (DCP; B-invariant). For JTK/RAIN/MH no closed form exists – simulation is required. 3. Simulation: run runSingleCohortPower() or runDifferentialPower(), or absorb a prior_result from a previous call to skip re-running.

Value

SCPRecommendResult with: $guidance – data.frame: method guidance table $analytical_df – data.frame[method, N, power_analytical] (DCP only) $simulation – runSingleCohortGrid() or runDifferentialPower() result (NULL if not run) $recommendation – data.frame[method, optimal_B, n_target, note]

---

plotSingleCohortPower

Plot single-cohort circadian power (3 panels)

Description

Plot single-cohort circadian power (3 panels)

Usage

```
plotSingleCohortPower(
  res,
  out_pdf = NULL,
  title = "",
  panel_a_res = NULL,
  panels = c("A", "B", "C"),
  fdr_thresholds = c(0.01, 0.05, 0.1, 0.2),
  panel_fdr = 0.05,
  vline_power = 0.8,
  vline_fdr = 0.2,
  fdr = NULL,
  p.adjust.method = "BH",
  reference_n = NULL,
  display_sizes = NULL,
  r_max = 5,
  cex_main = 1.5,
  cex_lab = 1.6,
  cex_axis = 1.45,
  line_lwd = 2,
  pt_cex = 0.95,
  legend_cex = NULL,
  title_cex = 1.65,
  oma_top = 2.4,
  legend_pos = "topleft",
  se_cap = 0.3,
  vertical = FALSE,
  panel_c_legend = FALSE,
  width = 15,
```

```
  height = 5.5
)
```

Arguments

res	Output from runSimsSingleCohort().
out_pdf	Path for PDF output. If NULL, plots to current device.
title	Overall figure title (default empty).
panel_a_res	Optional alternate result object used for the Panel A marginal-power computation only. If NULL (default), Panel A is computed from res. Useful when the marginal summary should reflect a different empirical assumption than the effect-size-stratified panels.
panels	Character vector, which of panels "A" (marginal power vs n), "B" (true discoveries by r-stratum), "C" (stratified power by r-stratum) to draw (default all three).
fdr_thresholds	FDR levels for panel A (default c(0.01,0.05,0.10,0.20)).
panel_fdr	FDR threshold highlighted in panels B/C (default 0.05).
vline_power	Horizontal reference line for target power (default 0.80).
vline_fdr	FDR threshold marked with a reference guide (default 0.20).
fdr	Single-value alias that sets both panel_fdr and vline_fdr at once (back-compat convenience; default NULL).
p.adjust.method	Correction method (default "BH").
reference_n	Reference sample size for annotations (default NULL = none).
display_sizes	Sample sizes to show in all panels (NULL = all).
r_max	Upper bound (r = A/sigma units) for the r-strata breakpoints; NULL adapts to the 95th percentile of observed r (default 5).
cex_main	Title character expansion (default 1.50).
cex_lab	Axis-label character expansion (default 1.60).
cex_axis	Axis-tick character expansion (default 1.45).
line_lwd	Line width for power curves (default 2).
pt_cex	Point character expansion for plotted markers (default 0.95).
legend_cex	Legend text character expansion; NULL (default) picks a size automatically.
title_cex	Outer-title character expansion (default 1.65).
oma_top	Outer margin (lines) reserved above the panels for the title (default 2.4).
legend_pos	Legend position keyword passed to legend() (default "topleft").
se_cap	Maximum half-width (power units) drawn for standard-error bars, to keep long bars from overplotting neighboring panels (default 0.3).
vertical	Logical; if TRUE, stack panels vertically instead of the default horizontal 1 x length(panels) layout (default FALSE).
panel_c_legend	Logical; if TRUE, also draw the legend on panel C (default FALSE; panel A's legend is normally sufficient).
width	PDF width in inches (default 15).
height	PDF height in inches (default 5.5).

Value

Invisibly returns list of data used per panel.

`plotDiffPower` *Plot differential circadian power*

Description

Default layout is marginal power vs sample size only (one column per endpoint, one row per comparison). The effect-size-stratified panels (power and true-discoveries by r) are opt-in via `stratified = TRUE`, which restores the original 3-column-per-row layout.

Usage

```
plotDiffPower(
  res_list,
  comp_labels = names(res_list),
  endpoints = c("DR", "DP", "DM"),
  fdr_thresholds = c(0.01, 0.05, 0.1, 0.2),
  panel_fdr = 0.05,
  vline_power = 0.8,
  vline_fdr = 0.2,
  display_sizes = NULL,
  stratified = FALSE,
  r_break_width = 0.25,
  r_max = 5,
  r_display_max = NULL,
  main_title = "Differential Circadian Power Analysis",
  out_pdf = NULL,
  width = NULL,
  height = NULL
)
```

Arguments

`res_list`	Named list of length 2, each element output of runSimsDiff().
`comp_labels`	Character(2), display labels for the two comparisons.
`endpoints`	Which endpoints to include (default c("DR","DP","DM")).
`fdr_thresholds`	FDR levels for panel A (default c(0.01,0.05,0.10,0.20)).
`panel_fdr`	FDR threshold used to draw the marginal power curve in the non-stratified layout (default 0.05).
`vline_power`	Horizontal reference line for target power, drawn at this value (default 0.80).
`vline_fdr`	FDR threshold marked with a reference guide on the power panels (default 0.20).
`display_sizes`	Sample sizes to show in all panels (NULL = all).

stratified	Logical (default FALSE). If FALSE, draws only the marginal power-vs-sample-size panel per endpoint (1 x n_endpoints layout per comparison). If TRUE, also draws power-by-r and true-discoveries-by-r strata panels in the original 3-column layout.
r_break_width	Width of r-strata bins (default 0.25). Ignored if stratified = FALSE.
r_max	Upper bound (in r = A/sigma units) for the r-strata breakpoints when stratified = TRUE (default 5).
r_display_max	Optional upper x-axis limit (r units) for the stratified panels; NULL (default) uses the full stratified range.
main_title	Outer plot title (default "Differential Circadian Power Analysis").
out_pdf	Path for PDF. If NULL, plots to current device.
width	PDF width in inches. Defaults adapt to stratified.
height	PDF height in inches. Defaults adapt to stratified.

Value

Invisibly, list of panel data per comparison/endpoint.

---

plotBootstrapDesignGrid

Plot bootstrap design grid results

Description

Panel A: Power vs N, one line per B, with bootstrap CI ribbon. Panel B: Heatmap of mean power over (N, B) grid.

Usage

```
plotBootstrapDesignGrid(
  result,
  test_type = "DR",
  fdr_threshold = NULL,
  panels = "A",
  output_file = NULL
)
```

Arguments

result	Output from runBootstrapDesignGrid()
test_type	Test type to plot
fdr_threshold	FDR threshold (label only)
panels	Which panel(s) to draw: "A" (power vs N) and/or "B" (heatmap) (default "A").
output_file	Path for PDF output (NULL = screen)

detect_cosinor *Unified cosinor rhythmicity detection (single- or multi-harmonic)*

Description

Canonical entry point for rhythmicity detection by K-harmonic cosinor regression. `K = 1` is the single-harmonic cosinor F-test, identical to `detect_DCP`; `K = 2` adds the 12-hour second harmonic and tests the joint harmonic block. By default the unshrunk exact nested F-test is used (matching the power derivations in the paper). Returns raw p-values; the caller applies FDR control.

Usage

```
detect_cosinor(expr, times, K = 1L, period = 24, ebayes = FALSE, mc.cores = 1L)
```

Arguments

`expr`	Gene x sample expression matrix
`times`	Numeric time vector (length = ncol(expr))
`K`	Harmonic order (default 1)
`period`	Period in hours (default 24)
`ebayes`	For `K >= 2`, moderate residual variance with empirical-Bayes shrinkage (default `FALSE`, the unshrunk exact F-test)
`mc.cores`	Cores for the `K >= 2` fit (default 1)

Value

Numeric vector of raw p-values, length `nrow(expr)`

See Also

`detect_DCP`

simCircadianSingleCohort2H *Simulate Single-Cohort Circadian Data Under the Two-Harmonic Cosinor Model*

Description

Two-harmonic analog of `simCircadianSingleCohort`. Generates expression from

$$y_{gi} = \mu_g + A_{g,1}\cos(\omega_0(t_i - \phi_{g,1})) + A_{g,2}\cos(2\omega_0(t_i - \phi_{g,2})) + \epsilon_{gi}$$

using the joint pilot tuple $(A_1, \phi_1, A_2, \phi_2, \sigma)$ stored on `bio.opts` by `estCircadianParam2H`. The same index is drawn across all five vectors so the pilot's cross-parameter dependence is preserved.

`r_values` is reported on the 1st-harmonic scale $r_g = A_{g,1}/\sigma_g$ to match the existing r-stratification machinery and the manuscript's r_g notation.

Usage

```
simCircadianSingleCohort2H(bio.opts, cts, seed = NULL)
```

Arguments

bio.opts	A CircadianBioOptions produced by estCircadianParam2H (must carry paired_2h = TRUE, amplitude2, phase2).
cts	Numeric vector of sample collection times in hours; length determines N.
seed	Optional integer random seed.

Value

Named list:

expr Gene expression matrix (ngenes x N).

is_rhythmic Logical vector.

r_values Per-gene 1st-harmonic SNR $A_{g,1}/\sigma_g$.

A1, phi1, A2, phi2 Per-gene truth vectors (length ngenes; zero for non-rhythmic genes).

See Also

estCircadianParam2H, simCircadianSingleCohort

---

makeAdaptiveRStrata

Compute adaptive r-strata breakpoints with fixed bin width

Description

Generates breakpoints from 0 to ceiling(r_max / bin_width) * bin_width, stepping by bin_width, then appends Inf. Every bin width is identical, but the number of bins adapts to the empirical r range of the pilot data.

Usage

```
makeAdaptiveRStrata(
  bio.opts,
  bin_width = 0.25,
  r_min_pct = 0,
  r_pctile_cap = 0.99
)
```

Arguments

bio.opts	CircadianBioOptions with amplitude and sigma_rhythmic.
bin_width	Width of each r bin (default 0.25).
r_min_pct	Lower percentile of pilot r used as range floor (default 0 = always start from 0).
r_pctile_cap	Upper percentile of pilot r used as the range ceiling before rounding up to the nearest bin_width (default 0.99).

Value

Numeric vector of breakpoints starting at 0 and ending at Inf.

`launchShiny` *Launch the SCP Shiny app*

Description

Opens the bundled simulation-based power-analysis app in the user's default browser. The app lets the user pick a bundled pilot (or upload their own), set target power and FDR, read off the recommended sample size, explore per-gene rhythmicity, and run pathway enrichment.

Usage

```
launchShiny(..., install_deps = TRUE, species_all = FALSE)
```

Arguments

`...`	Additional arguments passed to `shiny::runApp()`.
`install_deps`	Logical; if `TRUE` (default) install any missing optional app packages before launching. Set `FALSE` to skip the check (missing features then degrade gracefully inside the app).
`species_all`	Logical; if `TRUE` also install the annotation databases for rat, zebrafish, Drosophila, and Arabidopsis (in addition to human + mouse) so upload gene-symbol mapping works for every supported species. Default `FALSE` installs only human + mouse (the common case); the others install on demand or you can pass `TRUE` once.

Details

On first launch the function checks for the app's optional helper packages and offers to install any that are missing (so the gene-symbol mapping, XLSX export, full-app capture, and pathway-enrichment features work out of the box). These are kept out of the package's hard dependencies so a plain `install.packages("SCP")` stays lightweight and the API works without the large Bioconductor annotation databases.

Value

Invisibly returns `NULL`. Called for its side effect of launching the Shiny app.

Examples

```
## Not run:
  launchShiny()
  launchShiny(species_all = TRUE)     # also install rat/zebrafish/fly/plant DBs
  launchShiny(install_deps = FALSE)   # skip the dependency check

## End(Not run)
```

plot.SCPDiffResult *Plot an SCPDiffResult object*

Description

Plot an SCPDiffResult object

Usage

```
## S3 method for class 'SCPDiffResult'
plot(x, output_file = NULL, ...)
```

Arguments

x	An SCPDiffResult from runDifferentialPower().
output_file	Optional PDF path; NULL draws to the active device.
...	Ignored.

Value

x, invisibly.

plot.SCPRecommendResult *Plot an SCPRecommendResult object*

Description

Plot an SCPRecommendResult object

Usage

```
## S3 method for class 'SCPRecommendResult'
plot(x, output_file = NULL, ...)
```

Arguments

x	An SCPRecommendResult from recommendDesign().
output_file	Optional PDF path; NULL draws to the active device.
...	Passed to the underlying plot method.

Value

x, invisibly.

plot.SCPSingleResult *Plot an SCPSingleResult object*

Description

Plot an SCPSingleResult object

Usage

```
## S3 method for class 'SCPSingleResult'
plot(x, output_file = NULL, ...)
```

Arguments

x	An SCPSingleResult from runSingleCohortGrid().
output_file	Optional PDF path; NULL draws to the active device.
...	Ignored.

Value

x, invisibly.

print.CircadianAnalysisOptions *Print a CircadianAnalysisOptions object*

Description

Print a CircadianAnalysisOptions object

Usage

```
## S3 method for class 'CircadianAnalysisOptions'
print(x, ...)
```

Arguments

x	A CircadianAnalysisOptions object.
...	Ignored.

Value

x, invisibly.

print.CircadianBioOptions *Print a CircadianBioOptions object*

Description

Print a CircadianBioOptions object

Usage

```
## S3 method for class 'CircadianBioOptions'
print(x, ...)
```

Arguments

x	A `CircadianBioOptions` object.
...	Ignored.

Value

x, invisibly.

print.CircadianBootstrapOptions *Print a CircadianBootstrapOptions object*

Description

Print a CircadianBootstrapOptions object

Usage

```
## S3 method for class 'CircadianBootstrapOptions'
print(x, ...)
```

Arguments

x	A `CircadianBootstrapOptions` object.
...	Ignored.

Value

x, invisibly.

print.CircadianDesignOptions *Print a CircadianDesignOptions object*

Description

Print a CircadianDesignOptions object

Usage

```
## S3 method for class 'CircadianDesignOptions'
print(x, ...)
```

Arguments

x	A CircadianDesignOptions object.
...	Ignored.

Value

x, invisibly.

print.npower *Print an npower result*

Description

Print an npower result

Usage

```
## S3 method for class 'npower'
print(x, ...)
```

Arguments

x	An npower object (recommended n and its power curve).
...	Ignored.

Value

x, invisibly.

print.SCPDiffResult *Print an SCPDiffResult object*

Description

Print an SCPDiffResult object

Usage

```
## S3 method for class 'SCPDiffResult'
print(x, ...)
```

Arguments

x	An SCPDiffResult from runDifferentialPower().
...	Ignored.

Value

x, invisibly.

print.SCPRecommendResult *Print an SCPRecommendResult object*

Description

Print an SCPRecommendResult object

Usage

```
## S3 method for class 'SCPRecommendResult'
print(x, ...)
```

Arguments

x	An SCPRecommendResult from recommendDesign().
...	Ignored.

Value

x, invisibly.

`print.SCPSingleResult` *Print an SCPSingleResult object*

Description

Print an SCPSingleResult object

Usage

```
## S3 method for class 'SCPSingleResult'
print(x, ...)
```

Arguments

`x`	An `SCPSingleResult` from `runSingleCohortGrid()`.
`...`	Ignored.

Value

`x`, invisibly.

Index

CircadianAnalysisOptions, *1*, 15
CircadianBioOptions, *1*, 11
CircadianBootstrapOptions, *1*, 16
CircadianDesignOptions, *1*, 14
circaPowerApproxN80, *1*, 22

detect_cosinor, *1*, 28
detect_DCP, *28*

estCircadianParam, *1*, *3*, 5, *7*, *8*
estCircadianParam2H, *1*, 6, *28*, *29*
estCircadianParamFMM, *6*
estCircadianParamTwoGroup, *1*, *6*, 8

launchShiny, *1*, 30

makeAdaptiveRStrata, *1*, 29

npower, *1*, 21

plot.SCPDiffResult, 31
plot.SCPRecommendResult, 31
plot.SCPSingleResult, 32
plotBootstrapDesignGrid, *1*, 27
plotDiffPower, *1*, 26
plotSingleCohortPower, *1*, 24
prepCircadianData, *1*, *6*, 10
print.CircadianAnalysisOptions, 32
print.CircadianBioOptions, 33
print.CircadianBootstrapOptions, 33
print.CircadianDesignOptions, 34
print.npower, 34
print.SCPDiffResult, 35
print.SCPRecommendResult, 35
print.SCPSingleResult, 36

recommendDesign, *1*, 23
runBootstrapDesignGrid, *1*, 19
runDifferentialPower, *1*, 18
runSimsSingleCohort, *1*, 16

SCP *(SCP-package)*, 1
SCP-package, 1
scp_load_pilot, *1*, 3
scp_pilot_search, *1*, 4
scp_pilots, *1*, 2
simCircadianSingleCohort, *28*, *29*
simCircadianSingleCohort2H, *1*, *8*, 28